

Difference Between LINQ Cast and OfType Operators in C#

In this article, I will discuss the **Difference Between LINQ Cast and OfType Operators** in C# with Examples. This is one of the frequently asked interview questions. To better understand this question, I strongly recommend you read the following two articles before proceeding to this article.

1. [Cast Operator in C#](#)
2. [OfType Operator in C#](#)

Difference Between Cast and OfType operators:

The LINQ methods Cast and OfType in C# are used for converting collections to a specified type, but they differ in handling elements that cannot be converted to the target type.

The Cast operator in C# will try to cast all the elements in the source sequence into a specified type. If any of the elements in the source sequence cannot be cast to the specified type, it will throw an **InvalidCastException**.

On the other hand, the OfType operator in C# returns only the elements of the specified type, and the rest of the elements in the source sequence will be ignored and excluded from the result.

Key Differences:

- **Exception Handling:** Cast throws an exception for elements of the wrong type, while OfType excludes them without throwing an exception.
- **Intended Use:** Cast is used when the entire collection is expected to be of a single type, whereas OfType filters elements of a specified type from a mixed collection.

Cast Operator Example:

In the following example, the Cast operator throws **InvalidCastException** as it cannot convert the value "50" (in the form of a string) to an integer.

```
using System.Linq;
using System.Collections.Generic;
using System;
using System.Collections;
namespace ConversionOperators
{
    class Program
    {
        public static void Main()
        {
            ArrayList list = new ArrayList
            {
                10,
                20,
                30,
```

```

        "50"
    };
    IEnumerable<int> result = list.Cast<int>();
    foreach (int i in result)
    {
        Console.WriteLine(i);
    }
    Console.ReadKey();
}
}
}

```

Now run the above application; it should give you the following exception.



OfType Operator Example:

Let us rewrite the example using the **OfType** operator, as shown below.

```

using System.Linq;
using System.Collections.Generic;
using System;
using System.Collections;
namespace ConversionOperators
{
    class Program
    {
        public static void Main()
        {
            ArrayList list = new ArrayList
            {
                10,
                20,
                30,
                "50"
            };

            IEnumerable<int> result = list.OfType<int>();
            foreach (int i in result)
            {
                Console.WriteLine(i);
            }
            Console.ReadKey();
        }
    }
}

```

Output: 10 20 30

When you run the application, you will not get any exceptions. The value “50” is ignored and excluded from the result.

When Should We use Cast over OfType and Vice Versa?

LINQ Cast Method

- **Purpose:** The Cast method converts all elements in a non-generic collection (like ArrayList or IEnumerable) to a specified generic type.
- **Behavior:** It attempts to cast every element in the collection to the specified type.
- **Error Handling:** If any element cannot be cast to the specified type, Cast throws an InvalidCastException at runtime.
- **Use Case:** Use Cast when you are certain that every element in the collection can be cast to the specified type, and you want an exception if this is not the case.

We need to use the Cast Operator

1. When we want to cast all the elements in the collection
2. When we know for sure that the collection contains only elements of the specified type

LINQ OfType Method

- **Purpose:** The OfType method filters elements in the collection that are of or can be cast to a specified type.
- **Behavior:** It includes only those elements in the collection that can be cast to the specified type and silently ignores elements that cannot be cast.
- **Error Handling:** No exceptions are thrown if elements cannot be cast to the specified type; those elements are simply excluded from the result.
- **Use Case:** Use OfType when you want to safely extract elements of a specific type from a collection, ignoring elements of other types.

We need to use the OfType operator.

1. When we want to filter the elements and return only the specified type of elements.

Choose Cast and OfType based on whether the collection is expected to have a uniform type or if it contains a mix of types and you only need elements of a specific type.

In the next article, I will discuss the [Difference Between LINQ Join and Group Join Methods in C#](#) with Examples. In this article, I explain the **Difference Between LINQ Cast and OfType Operators in C#** with Examples, and we also discussed when to use Cast and OfType operators in C# with Examples.